

HACKSIMUL8 2026 — Problem Statement Analysis and Solution Plan

Problem: 6-DOF UAV Quadcopter — modelling, PID altitude control, and ML-based self-tuning of PID gains
Reference cited by the organisers: Mien, T. & Tu, T. (2024), "*Design and Quality Evaluation of the Position and Attitude Control System for 6-DOF UAV Quadcopter Using Heuristic PID Tuning Methods*", IJRCS 4(4), 1712–1730 **Department:** Mechanical Engineering, PES University · 8-hour hackathon

What is actually being asked

Task	Ask	Where the marks are
1	Dynamic mathematical model of the 6-DOF quadcopter in MATLAB and Simulink, expressed as state-space equations	Correct equations, correct state vector, and a <i>demonstration</i> that the model behaves physically
2	Altitude control system using PID	" <i>Neat description of the methodology used for tuning... Use the best method available.</i> " The tuning method is graded, not just the gains
3	Generate data from simulation experiments and train an ML model for self-tuning of PID gains	The dataset design — what are the features, what are the labels
4	Implement the ML model, present test cases with suitable comparisons and merits	Honest comparison, quantified, including limitations

Read Task 2's wording again. The requirement is explicitly about *methodology*. A team that tunes by dragging sliders and shows a nice plot will lose to a team that explains *why* they chose their method. That is the single biggest scoring opportunity in this problem.

Table 1 — parameters (extracted from the PDF)

Parameter	Symbol	Value
Quadcopter mass	m	0.516 kg
Arm length	l	0.225 m
Gravity	g	9.81 m/s ²
Inertia moment of the rotor	I_M	3.368×10 ⁻⁵ kg·m ²
Thrust factor of rotor	k	2.996×10 ⁻⁶ N·s ²
Drag coefficient	b	1.260×10 ⁻⁷ N·m·s ²
Inertial constants	I_xx, I_yy	4.984×10 ⁻³ kg·m ²
	I_zz	8.958×10 ⁻³ kg·m ²

Derived: weight **W = mg = 5.062 N**; hover speed of each rotor **$\omega_{\text{hover}} = \sqrt{(W/4k)} = 649.9 \text{ rad/s}$** .

These are already coded in `solution/quad_params.m`.

Geometry from Fig. 1

A **cross (plus) configuration**: the arms lie along the body axes, with rotor 1 on +x_B, rotor 2 on -y_B, rotor 3 on -x_B, rotor 4 on +y_B. Rotors 1 and 3 spin one way, 2 and 4 the other, so their reaction torques cancel in hover.

Thrust per rotor $f_i = k \cdot \omega_i^2$, reaction torque $\tau_i = b \cdot \omega_i^2$. Taking $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$ for each arm gives the four virtual inputs:

$U1 = k(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2)$	total thrust	[N]	
$U2 = k(\omega_4^2 - \omega_2^2)$	roll input	[N]	→ torque = 1·U2
$U3 = k(\omega_3^2 - \omega_1^2)$	pitch input	[N]	→ torque = 1·U3
$U4 = b(\omega_1^2 + \omega_3^2 - \omega_2^2 - \omega_4^2)$	yaw torque	[N·m]	

Check the signs against Fig. 1 before you present. If roll responds backwards, flip the sign of U2. This is the most common quadcopter sign error and takes ten seconds to test with a step.

Task 1 — the model

State vector (12 states):

$$\mathbf{x} = [X \ \dot{X} \ Y \ \dot{Y} \ Z \ \dot{Z} \ \phi \ \dot{\phi} \ \theta \ \dot{\theta} \ \psi \ \dot{\psi}]^T$$

Nonlinear equations (implemented in `quad_dynamics.m`):

$$\begin{aligned}\ddot{X} &= (U1/m)(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ \ddot{Y} &= (U1/m)(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ \ddot{Z} &= -g + (U1/m) \cos \phi \cos \theta \\ \phi'' &= ((I_{yy} - I_{zz})/I_{xx}) \theta \dot{\psi} - (I_M/I_{xx}) \dot{\theta} \Omega_r + (1/I_{xx}) U2 \\ \theta'' &= ((I_{zz} - I_{xx})/I_{yy}) \phi \dot{\psi} + (I_M/I_{yy}) \dot{\phi} \Omega_r + (1/I_{yy}) U3 \\ \psi'' &= ((I_{xx} - I_{yy})/I_{zz}) \phi \dot{\theta} + (1/I_{zz}) U4\end{aligned}$$

with $\Omega_r = \omega1 - \omega2 + \omega3 - \omega4$ the residual rotor speed driving the gyroscopic term.

Linearised state space about hover (ϕ, θ, ψ small; $U1 = W + \delta U1$). The system decouples into four independent double integrators:

Channel	Equation	Transfer function
Altitude	$\ddot{Z} = \delta U1/m$	$1/(m s^2)$
Roll	$\phi'' = (1/I_{xx}) U2$	$1/(I_{xx} s^2)$
Pitch	$\theta'' = (1/I_{yy}) U3$	$1/(I_{yy} s^2)$
Yaw	$\psi'' = (1/I_{zz}) U4$	$1/(I_{zz} s^2)$

Plus the position coupling $\ddot{X} = g \cdot \theta$ and $\ddot{Y} = -g \cdot \phi$ — the quadcopter translates by tilting, which is why position control needs an inner attitude loop.

All 12 open-loop poles sit at the origin. Say this out loud to the judges: the quadcopter is marginally stable at best and *cannot* fly without feedback. It motivates everything that follows.

`task1_statespace.m` builds A, B, C, D analytically **and** verifies them by finite- differencing the nonlinear model. Showing $\max |A - A_{\text{num}}| \approx 1e-9$ is hard evidence your model is right — far more convincing than asserting it.

Task 2 — altitude PID, and the tuning methodology

The key design move: feedback linearisation

The true altitude dynamics are nonlinear and coupled to attitude:

$$\ddot{Z} = -g + (U1/m) \cos \phi \cos \theta$$

Rather than linearising and hoping the angles stay small, **cancel the nonlinearity exactly**. Command the thrust as

$$U1 = m (g + u_z) / (\cos \phi \cos \theta)$$

Substituting gives, exactly and for any attitude:

$$\ddot{Z} = u_z$$

The PID now sees a clean double integrator $1/s^2$, and the design stays valid at large tilt — not just near hover. The `mg` term is gravity feed-forward; the $1/(\cos \cdot \cos)$ term restores the thrust lost to tilting.

This is the headline of your Task 2 slide.

The four tuning methods (all implemented and compared)

Method	What it does	Why include it
Ziegler–Nichols	<i>Fails here</i>	For a pure double integrator, P-only control oscillates at every K_p , so there is no unique ultimate gain K_u . Saying this proves you understood the plant instead of applying a recipe.
A — Analytical pole placement	$s^2 + K_d s + K_p = 0$ matched to $s^2 + 2\zeta\omega_n s + \omega_n^2$ gives $K_p = \omega_n^2$, $K_d = 2\zeta\omega_n$	Principled starting point, closed form, from a stated spec ($\zeta = 0.9$, $T_s = 2$ s)
B — <code>pidtune</code>	MATLAB's automatic tuner on the linear plant	Fast, gives phase margin and crossover for free
C — <code>pidtune</code> with target <code>wc</code>	Same, with the bandwidth specified	Shows you can trade speed against stability deliberately
D — ITAE optimisation	<code>fminsearch</code> minimising ITAE on the nonlinear model with saturation and anti-windup active	This is "the best method available" — the other three optimise an approximation, this one optimises what actually flies

Method D is the honest answer to the organisers' instruction. Present the progression $A \rightarrow B \rightarrow C \rightarrow D$ as a narrative: each step relaxes an assumption the previous one made.

Realism that earns credibility

`sim_altitude.m` includes all of these, and you should point them out:

- **Actuator saturation** — thrust clipped to what the rotors can physically produce
- **Clamping anti-windup** — the integrator stops accumulating while saturated
- **Derivative on measurement**, not on error — no derivative kick on a step command
- **Discrete controller at 100 Hz** with RK4 plant integration, as real hardware would run

A control demo with no actuator limit is not a control demo. Judges ask about this.

The plot that sets up Task 3

The last figure in `task2_altitude_pid.m` runs the *same* fixed gains at mass $\times 1.0$, $\times 1.4$, $\times 1.8$, $\times 2.2$. Performance visibly degrades. **That plot is the reason Task 3 exists** — show it as the bridge between the two tasks.

Task 3 — the ML self-tuner (the part most teams will get wrong)

Two framings — pick the right one

(a) Gain scheduling. Features = the operating condition (mass, wind). The controller must be *told* the payload mass. Weak: in flight you do not know it. Most teams will do this without noticing the flaw.

(b) Response-signature inference. Fly briefly with a baseline PID, **measure how the aircraft actually responded**, and infer the right gains from that signature. The controller discovers the mass and disturbance from behaviour.

We implement (b). The network learns the inverse map

observed closed-loop response → optimal PID gains

If a judge asks what makes your approach self-tuning rather than scheduled, this is the answer, and it is a strong one.

Dataset design

- **300 sampled operating conditions:** mass ratio 1.0–2.4, steady wind ± 1 N, gust 0–1.5 N, reference step 0.5–2.0 m, sustained tilt 0–0.35 rad.
- For each condition: 1. Fly with the **baseline** gains → extract the **8-feature signature** 2. Run ITAE optimisation → the **optimal gains** for that condition (the label)

Features (all measurable in flight — this is the point):

#	Feature	Why it carries information
1	overshoot	reveals effective damping
2	rise time	reveals effective bandwidth
3	settling time	reveals damping and lag together
4	normalised steady-state error	reveals disturbance and integral shortfall
5	normalised IAE	overall tracking quality
6	mean thrust / nominal weight	directly reveals the mass ratio
7	peak thrust / weight	reveals thrust margin remaining
8	commanded step size	the manoeuvre is part of the context

Feature 6 is the one to point at. In steady hover the thrust equals the weight, so the measured thrust ratio *is* the mass ratio. The network does not have to be told the payload — it can read it off. That is a genuinely nice piece of physical insight, and it is worth saying explicitly.

Labels: the optimal `[Kp, Ki, Kd]` for that condition.

The model

A shallow feedforward network, `fitnet([12 8])` trained with Levenberg–Marquardt (Deep Learning Toolbox — the toolbox the organisers named). 70/15/15 train/validation/test split. The script falls back to

linear least squares if the toolbox is missing, so it runs either way.

Reported per gain: RMSE, MAE, R^2 . Plus predicted-vs-true scatter plots with the ideal diagonal.

Why a small network and not a deep one: 300 samples, 8 features, a smooth underlying map. A deep network would overfit. Choosing the right *size* of model for the data is itself a defensible engineering decision — say so rather than apologising for it.

Task 4 — implementation, test cases, comparison

How the self-tuner runs

```
Phase 1 (0 → 3 s)  fly with the fixed baseline gains, record the response
At 3 s              compute the signature → network → new gains
                    hand the integrator state over (bumpless transfer)
Phase 2 (3 s → 12 s) continue with the predicted gains
```

Predicted gains are **clamped** to a safe range before use. Say this — it is the answer to "what if the network outputs something destabilising?"

Five test cases, deliberately outside the training grid

1. Heavy payload ($m \times 1.8$)
2. Strong wind + mid-flight gust
3. Large step + sustained 0.3 rad tilt
4. Combined worst case
5. Sensor noise + payload

The three-way comparison — this is what makes it convincing

For every test case, compare:

Controller	Meaning
Fixed PID	the Task 2 baseline — the thing you are trying to beat
ML self-tuned	your contribution
Oracle optimal	gains optimised directly for that exact condition — the theoretical best achievable

Beating the fixed PID is expected. **Showing how much of the gap to the oracle you close** is what separates a good result from a hand-wave. Report mean ITAE improvement over fixed, and mean remaining gap to oracle.

State the limitations (Task 4 asks for "merits" — give both)

- Valid only inside the sampled envelope; outside it the network extrapolates
- Needs a 3 s identification window before it can adapt

- Trained on simulation, so it inherits every modelling error (rigid airframe, no blade flapping, no motor lag)
- No formal stability guarantee — hence the output clamp

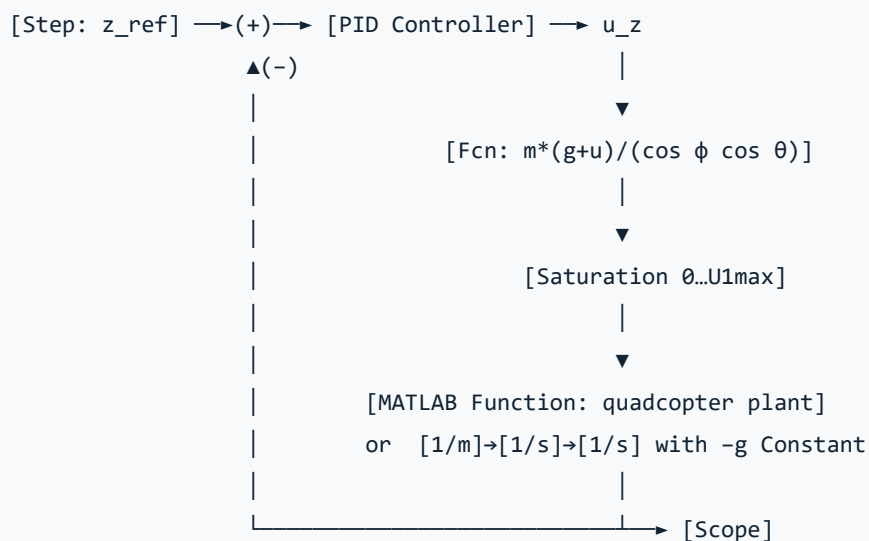
Judges reward honesty here far more than they punish the limitation.

The Simulink deliverable — do not skip this

This is a **MATLAB and Simulink** hackathon run by the Mechanical department. A scripts-only submission will be marked down however good the results are. Build the model.

Minimum viable Simulink model (about 45 minutes)

`quad_altitude.slx`:



Build steps:

1. New model. Set solver **fixed-step ode3**, step size `0.001`, stop time `12`.
2. Add **PID Controller** block (Simulink → Continuous). Set Kp/Ki/Kd to the workspace variables from Task 2. Enable **Limit output** and **anti-windup: clamping**.
3. Feedback linearisation: a **Fcn** block or MATLAB Function computing `m*(g+u)/(cos(phi)*cos(theta))`.
4. **Saturation** block with limits `P.U1_min`, `P.U1_max`.
5. Plant: either a **MATLAB Function** block wrapping `quad_dynamics`, or build the vertical channel from blocks — `Gain(1/m)` → `Sum` (with `-g` Constant) → `Integrator` → `Integrator`.
6. **Set every parameter from a workspace variable**. Load them with `P = quad_params();` in the model's **PreLoadFcn** callback.
7. Log `z`, `U1` and `z_ref` to the **Simulation Data Inspector**.

For Tasks 3–4 in Simulink: put the trained network in a **MATLAB Function** block (or the **Predict** block from Deep Learning Toolbox), and use a **Stateflow chart** with two states — `Identify` and `Adapted` — with the transition `after(3, sec)`. That chart is a neat, visible way to show the two-phase logic, and it uses the third recommended course.

Time plan for the day

Time	Do	Owner
now → 10:00	Install MATLAB; run <code>task1_statespace.m</code> ; verify the model	whole team
10:00 → 11:00	Build <code>quad_altitude.slx</code> in Simulink	member A
10:00 → 11:30	Run <code>task2_altitude_pid.m</code> , pick gains, capture plots	member B
11:30 → 13:00	Start <code>task3_generate_data_train_ml.m</code> (it is the long one — start it early and let it run over lunch)	member C
13:00 → 13:45	Lunch, while Task 3 trains	—
13:45 → 15:00	<code>task4_test_ml_selftuning.m</code> ; Stateflow chart in Simulink	all
15:00	FREEZE. No new features.	—
15:00 → 16:15	Slides, figures, numbers, rehearse the walkthrough twice	all
16:15 → 16:30	Evaluation	—

Start Task 3 before lunch. It takes 5–12 minutes of pure compute, and if it fails you want to discover that at 11:30, not at 15:00.

The seven numbers to have on a slide

1. Settling time (s) — fixed vs ML
2. Percent overshoot — fixed vs ML
3. Steady-state error (m)
4. ITAE — fixed vs ML vs oracle
5. Phase margin (deg) and gain margin (dB) of the altitude loop
6. Peak thrust vs saturation limit
7. Mean ITAE improvement (%) and remaining gap to oracle (%)

Questions the judges will ask, and your answers

"Why is the altitude plant a double integrator?" Newton's second law integrated twice: thrust → acceleration → velocity → position. Both poles at the origin, which is why proportional control alone cannot stabilise it and derivative action is mandatory.

"Why not just use Ziegler–Nichols?" It is undefined for this plant. P-only control of a pure double integrator oscillates at every gain, so there is no unique ultimate gain K_u to measure.

"What operating point did you linearise around?" Hover: all angles zero, all rates zero, $U_1 = mg = 5.062$ N. But the altitude controller uses feedback linearisation, so it stays valid away from hover too.

"What happens when the actuator saturates?" Thrust is clipped to the rotors' physical limit and clamping anti-windup stops the integrator accumulating. Both are in the model, and the thrust trace shows the limit.

"How is this self-tuning and not gain scheduling?" The controller is never told the mass or the wind. It flies for 3 s, measures its own response, and infers the gains from that signature — in particular, mean thrust divided by nominal weight directly reveals the mass ratio.

"How do you know the ML gains are any good?" We compare against an oracle that optimises directly for each test condition. That bounds the best achievable, and we report how much of the gap we close.

"What would you do with more time?" Close the position and attitude loops (the cascade the reference paper builds), then extend the self-tuner to all four channels rather than altitude alone.

Files

File	Purpose
<code>solution/quad_params.m</code>	Table 1 parameters + derived quantities
<code>solution/quad_dynamics.m</code>	Full nonlinear 6-DOF model (Task 1)
<code>solution/rotor2U.m</code>	Rotor speeds → virtual inputs, with the geometry
<code>solution/task1_statespace.m</code>	State space, linearisation check, controllability
<code>solution/task2_altitude_pid.m</code>	Four tuning methods compared (Task 2)
<code>solution/sim_altitude.m</code>	Nonlinear closed-loop simulator, saturation + anti-windup
<code>solution/perf_metrics.m</code>	T_r , T_s , OS, SSE, ITAE, IAE, ISE, control effort
<code>solution/alt_cost.m</code>	ITAE cost for gain optimisation
<code>solution/task3_generate_data_train_ml.m</code>	Dataset generation + NN training (Task 3)
<code>solution/task4_test_ml_selftuning.m</code>	Self-tuner, 5 test cases, comparisons (Task 4)
<code>solution/run_all.m</code>	Runs everything in order